

ECE2.414 – Advanced Memory Circuits and Systems

Assignment 1: Simulating Memory – Devices to Systems

Abhinav Venkata Kota

IIIT Hyderabad · Monsoon 2026

Part A – ngspice: 6T SRAM Read Margin

Central question: does the bitcell read correctly at 0.7 V and 85°C?

Task	Result	Interpretation
1	$\Delta V(BL, BLB)$ at 2 ns = 729.86 mV; $q_{\max} = 205.52$ mV.	Reads correctly; margin is $\sim 10.6\times$ the 69 mV lecture reference.
2	Access width 0.24 μm : $\Delta V = 852.34$ mV, $q_{\max} = 271.22$ mV.	Read disturb grows; cell is more marginal, not stable enough for worse PVT corners.
3	ΔV falls below 25 mV at $V_{DD} \approx 0.448$ V.	Within 1.1–0.6 V sweep, read still safe; failure starts below ~ 0.45 V.
4	At 85 °C: $\Delta V = 582.12$ mV, $q_{\max} = 220.76$ mV.	ΔV moves more than sense-amp offset; limiting failure mode is read disturb.

Table 1: Part A summary of nominal, sizing, supply, and temperature results.

Task 1: Nominal Read Margin

The bitline differential $\Delta V(BL, BLB)$ measured at $t = 2$ ns is 729.86 mV, while the stored-node voltage $v(Q)$ rises to a maximum of $q_{\max} = 205.52$ mV before settling. Because the sense amplifier only requires ~ 69 mV of differential to resolve correctly (the lecture reference value), the nominal margin is roughly $10.6\times$ larger than the minimum needed. This large margin means the cell reads correctly at nominal conditions with substantial headroom against process, voltage, and temperature (PVT) variation. The reason the margin is so generous is that at the nominal $V_{DD} = 1.1$ V, the pull-down NMOS devices (MD1/MD2) are much stronger than the access transistors (MA1/MA2), so the voltage divider formed during read keeps $v(Q)$ well below the inverter trip point of the cross-coupled pair.

Task 2: Access-Device Sizing

Widening the access transistors from the nominal width to 0.24 μm strengthens the path between the bitline and the internal storage node. This has two effects: (i) the bitline differential increases to 852.34 mV because the wider device discharges BL faster, and (ii) the stored-node disturbance $q_{\max}$ rises from 205.52 mV to 271.22 mV – a 31.9% increase – because more current flows into the “0” storage node during the read, pushing it closer to the inverter trip point. Although the cell still survives at nominal PVT, the enlarged disturb erodes the static noise margin (SNM). At worst-case process corners (slow NMOS, fast PMOS) or elevated temperature, this reduced margin could cause a destructive read, flipping the stored bit. The trade-off is fundamental: wider access devices improve sensing speed and bitline swing but degrade read stability. SRAM designers resolve this tension through careful cell ratio (β -ratio) tuning, where the pull-down device must always be stronger than the access device.

Task 3: Supply Sweep

Sweeping V_{DD} from 1.1 V down to 0.4 V reveals that ΔV decreases monotonically with supply voltage. The 25 mV sense-amplifier offset threshold is crossed at approximately 0.448 V (obtained by linear interpolation between the 0.45 V and 0.40 V data points). Within the requested 1.1–0.6 V window, the minimum ΔV is 143.9 mV (at 0.6 V), which remains $\sim 5.8\times$ above the offset threshold, so the read works safely across

this entire range. The monotonic decline occurs because lower V_{DD} reduces the overdrive voltage on both the access and pull-down transistors, weakening the current that develops the bitline differential. The near-linear relationship between ΔV and V_{DD} in the mid-range reflects the fact that both transistor stacks scale roughly proportionally with supply.

V_{DD} (V)	1.10	1.05	1.00	0.95	0.90	0.85
ΔV (mV)	729.9	673.6	616.0	557.1	497.4	437.2
V_{DD} (V)	0.80	0.75	0.70	0.65	0.60	0.55
ΔV (mV)	376.8	316.6	257.1	199.1	143.9	94.2
V_{DD} (V)	0.50	0.45	0.40			
ΔV (mV)	53.6	25.7	10.7			

Table 2: Part A Task 3 – supply sweep. Linear interpolation between 0.45 V and 0.40 V gives the 25 mV crossing near 0.448 V.

Task 4: Temperature Sensitivity

At 85 °C, ΔV drops to 582.12 mV (a 20.2% reduction from nominal) and $q_{\max}$ rises to 220.76 mV (a 7.6% increase). The drop in ΔV occurs because elevated temperature reduces carrier mobility in the MOSFET channel, lowering the drive current that develops the bitline differential. Simultaneously, the higher temperature increases subthreshold leakage, which allows more current to flow into the stored node during the read access, raising $q_{\max}$. The fact that ΔV moves more than the sense-amplifier offset means that the dominant failure mechanism at high temperature is read disturb (i.e., $q_{\max}$ approaching the trip point), not the sense amplifier’s inability to resolve the differential. This is the expected hierarchy for 6T SRAM: at elevated temperatures, the cell’s internal stability degrades faster than the external sensing path.

The nominal access-device result serves as the reference point for the later system-level comparison. Widening MA1/MA2 increases $q_{\max}$ by 31.9%, the high-temperature differential is 20.2% below nominal, and stored-node disturbance is 7.6% higher. The voltage margin remains large at the stated operating point, but both sizing and temperature push the stored state in the unfavorable direction – a fact that becomes important when selecting technology in Part E.

Part B – CACTI: 2 MB SRAM L2

Central question: how fast, how big, and how leaky is a 2 MB SRAM at 45 nm?

Metric	Value	Sweep / Alternative	Value
Access time	2.90184 ns	256 kB $\rightarrow$ 16 MB	2.417 $\rightarrow$ 6.362 ns
Read energy	0.792885 nJ	Pure delay / area / ED ²	4/2/1, 2/2/1, 4/2/1
Leakage power	562.632 mW	Best organization	4/2/1
Area	11.4742 mm ²	Bitline hand-check	0.4069 vs 18.7 ns

Table 3: Part B baseline CACTI result and related reference values.

Task 1: Baseline Organization

CACTI reports a 2 MB SRAM L2 cache at 45 nm with the following baseline metrics: 2.90184 ns access time, 0.792885 nJ read energy per access, 562.632 mW total leakage power, and 11.4742 mm² die area. The optimal organization is Ndwl/Ndbl/Nspd = 4/2/1, meaning the data array is subdivided into 4 wordline divisions and 2 bitline divisions with a single subarray per data-out bit. This organization reflects CACTI’s internal optimizer balancing decoder delay, bitline length, wordline RC, and H-tree

routing to find the partition that minimizes access time. The 4/2/1 split provides enough mat-level parallelism to keep the wordline and bitline segments short, reducing their RC delay contribution, while not over-subdividing to the point where peripheral overhead (decoders, drivers, muxes) dominates area and energy.

Task 2: Capacity Scaling

Capacity	Access (ns)	Read energy (nJ)	Leakage (mW)	Area (mm ²)
256 kB	2.41714	0.658021	90.2876	4.96464
512 kB	2.48451	0.678967	158.126	5.94616
1 MB	2.63376	0.716643	293.680	7.59985
2 MB	2.90184	0.792885	562.632	11.4742
4 MB	3.53084	0.930134	1102.31	18.4579
8 MB	4.47269	1.21975	2183.82	39.4961
16 MB	6.36235	1.74213	4244.52	74.6677

Table 4: Part B Task 2 – capacity sweep. All metrics grow with capacity, but at different rates.

Access time rises from 2.417 ns at 256 kB to 6.362 ns at 16 MB – a $2.63\times$ increase for a $64\times$ capacity growth. The “one gate delay per capacity doubling” heuristic holds only loosely: doubling from 2 MB to 4 MB adds 21.7% to access time, while doubling from 4 MB to 8 MB adds 26.7%. The super-linear growth at larger capacities occurs because wire delay (which scales with physical distance, not transistor count) and H-tree routing overhead become the dominant latency components. Leakage power scales almost linearly with capacity because each additional bitcell contributes a roughly constant subthreshold current. Area scaling is also near-linear, though peripheral circuits introduce a sub-linear component at smaller sizes.

Task 3: Organization Objective

Objective	Access (ns)	Best Ndwl/Ndbl/Nspd	Interpretation
Pure delay	2.89231	4/2/1	More mat parallelism minimizes critical path.
Pure area	3.35518	2/2/1	Fewer subdivisions reduce peripheral overhead.
ED ²	2.90184	4/2/1	Delay benefit dominates the energy–delay trade-off.

Table 5: Part B Task 3 – objective comparison.

Pure delay and ED² both select the 4/2/1 organization, while pure area selects 2/2/1. The reason is intuitive: subdividing the array into more mats (higher Ndwl) shortens wordline segments and reduces their RC delay, directly lowering access time. However, each subdivision adds its own decoder, sense amplifier, and driver circuitry, increasing total area. When the optimizer targets area, it prefers fewer subdivisions (2/2/1) to minimize this peripheral overhead, accepting a 16% access-time penalty (3.35 ns vs. 2.89 ns). The fact that ED² agrees with pure delay indicates that at this capacity and technology node, the delay reduction from additional parallelism more than compensates for the modest energy increase from extra peripheral logic.

Task 4: Bitline Delay Check

CACTI reports a bitline delay component of 0.4069 ns. The lecture RC formula ($t = 0.69 \cdot R_{BL} \cdot C_{BL}$, using the full bitline resistance and capacitance) yields 18.7 ns – a $46\times$ overestimate. This dramatic discrepancy arises because the hand model treats the read as a full-swing passive RC discharge from V_{DD} to ground, whereas the actual SRAM read is a *partial-swing* operation terminated early by a sense amplifier.

The sense amplifier fires as soon as the bitline differential exceeds its input-referred offset (typically 25–50 mV), so the bitline only needs to swing a small fraction of V_{DD} . Additionally, CACTI accounts for the hierarchical bitline architecture (local and global bitlines with intermediate sense amplifiers), which keeps the effective RC product per segment much smaller than the full-column value. The capacity sweep corroborates this: as capacity grows, wire length and H-tree loading become increasingly dominant, which the simple RC model cannot capture since it ignores the hierarchical organization entirely.

Part C – NVSim: 2 MB STT-MRAM vs SRAM

Central question: same capacity, same periphery, different bitcell – what changes?

Metric	SRAM	STT-MRAM	Ratio
Area	6.393 mm ²	3.590 mm ²	0.56×
Read hit latency	0.850 ns	1.591 ns	1.87×
Write latency	0.520 ns	10.608 ns	20.4×
Read energy	0.575 nJ	0.763 nJ	1.33×
Write energy	0.531 nJ	0.435 nJ	0.82×
Leakage power	3196 mW	425 mW	0.13×

Table 6: Part C Task 1 – SRAM vs. STT-MRAM at 2 MB, 45 nm.

Task 1: SRAM–STT-MRAM Comparison

STT-MRAM achieves 0.56× the area and 0.13× the leakage power of SRAM, but at the cost of 1.87× read latency and 20.4× write latency. The area advantage stems from the 1T-1MTJ bitcell, which occupies roughly 6–8 F² compared to SRAM’s ~120–150 F² for a 6T cell. The massive leakage reduction occurs because the MTJ stores data as a magnetic orientation rather than as charge on a cross-coupled latch – there are no continuously biased transistors holding state. The read-latency penalty arises because sensing the resistance difference between parallel and anti-parallel MTJ states requires a current-mode sense amplifier with a longer integration time than the voltage-mode sensing in SRAM. The write-latency penalty is even more severe because STT switching requires injecting a spin-polarized current pulse of sufficient magnitude and duration to flip the free-layer magnetization; the thermal stability factor (Δ) sets a minimum pulse width to ensure a low write-error rate.

Task 2: Strengths and Weaknesses

Config	Area (mm ²)	Rd lat. (ns)	Wr lat. (ns)	Rd E (nJ)	Wr E (nJ)	Leakage (mW)
SRAM baseline	6.393	0.850	0.520	0.575	0.531	3196.1
STT-MRAM	3.590	1.591	10.608	0.763	0.435	425.2
TMR high	3.580	1.596	10.657	0.761	0.433	425.2
Half reset current	3.590	1.571	10.588	0.762	0.433	425.0

Table 7: Part C Tasks 1, 3, and 4 – NVSim variants. E denotes dynamic access energy.

The worst metrics for STT-MRAM are read and write latency; the best are area and leakage. At the device level, this is a direct consequence of the storage mechanism: SRAM stores data as charge on bistable inverters that can be read and written at transistor speeds, while STT-MRAM relies on spin-transfer torque to manipulate a magnetic tunnel junction. The current-mode sensing path for MRAM is inherently slower than voltage-mode sensing because the resistance ratio (TMR) provides a smaller signal margin, and the write path requires sustained current above the critical switching threshold for a duration set by

the thermal stability requirements. Read energy is higher for MRAM because the sense amplifier must drive current through the MTJ's resistance, whereas SRAM sensing exploits passive charge sharing on pre-charged bitlines.

Task 3: TMR Sensitivity

Raising the tunnel magnetoresistance ratio from 2:1 ($R_{AP}/R_P = 3k\Omega/6k\Omega$) to 3:1 ($4k\Omega/12k\Omega$) produces negligible changes in array-level metrics: the largest shift is write latency at only +0.46%. This is because TMR primarily affects the *signal margin* available to the sense amplifier – a higher TMR means a larger resistance difference between the “0” and “1” states, which improves the bit-error rate (BER) and allows more robust sensing. However, it does not fundamentally change the speed of either the sensing circuit (which is dominated by the amplifier's bandwidth and the bitline RC) or the write operation (which is governed by the spin-torque switching dynamics, independent of TMR). In practice, the benefit of higher TMR manifests as improved yield and reliability at the array level, not as a performance improvement – an important distinction for MRAM technology scaling.

Task 4: Write-Current Sensitivity

Halving the reset current from $200\mu A$ to $100\mu A$ does not change the cell area in NVSim because the MTJ's physical footprint ($54F^2$) dominates the total cell area, not the access transistor. The access transistor must be large enough to source the switching current, but at $200\mu A$ the transistor is already smaller than the MTJ pillar and its surrounding keep-out zones. Reducing the current to $100\mu A$ would allow a smaller transistor, but since the cell layout is MTJ-area-limited rather than transistor-limited, NVSim reports no area change. In a hypothetical design where the transistor were the area-limiting element (e.g., at a more advanced technology node with a smaller MTJ), halving the write current would enable a proportionally smaller transistor and a measurable area reduction. The slight reductions in latency (−1.3%) and energy (−0.5%) reflect the lower $I \cdot t$ product needed for switching.

The quantitative trade-off is clear: STT-MRAM reduces area by 43.8% and leakage by 86.7%, but read latency increases by 87.2% and write latency by $20.4\times$. The write-energy reduction is only 18.1%. The primary advantage of STT-MRAM is therefore density and standby power – it enables a much larger cache in the same die area, which becomes the key lever exploited in Part E.

Part D – Ramulator 2: DDR4 Miss Stream

Central question: how do scheduler, address mapping, and channel count change latency and throughput?

Configuration	Cycles	Avg read lat.	Row-hit rate	Note
FRFCFS + RoBaRaCoCh	9 594	262.93	53.9%	Baseline
FCFS-proxy (buf=1)	42 937	61.90	69.6%	No reordering; much slower
ChRaBaRoCo	78 845	1844.21	55.9%	Row locality destroyed
2 channels	4 889	233.57	32.7%	Best throughput

Table 8: Part D Task 1 – scheduler, address-mapping, and channel comparison.

Task 1: Baseline Scheduler and Mapping

With the First-Ready First-Come-First-Served (FRFCFS) scheduler and the RoBaRaCoCh address mapping, Ramulator 2 completes the trace in 9 594 cycles with an average read latency of 262.93 cycles and a 53.9% row-buffer hit rate. FRFCFS achieves this by prioritizing requests that hit an already-open row (“first-ready”) over older requests that would require a precharge–activate sequence, effectively

reordering the request stream to exploit spatial locality within DRAM rows. The RoBaRaCoCh mapping places row bits in the most significant address positions, which means consecutive cache-line addresses map to different columns within the same row, maximizing row-buffer hits for sequential access patterns.

Configuration	Cycles	Read lat.	Hits	Misses	Conflicts	Hit rate
FRFCFS + RoBaRaCoCh	9 594	262.93	1 079	215	707	53.9%
FCFS proxy	42 937	61.90	–	–	–	69.6%
ChRaBaRoCo	78 845	1844.21	1 121	7	878	55.9%
2 channels	4 889	233.57	326	143	528	32.7%

Table 9: Part D – row-buffer and throughput breakdown across configurations.

Task 2: Scheduler Comparison

The FCFS-proxy configuration (buffer depth = 1, no refresh) is not a true FCFS scheduler but a serializing proxy that processes one request at a time without reordering. Despite achieving a higher row-hit rate (69.6%) – because it processes requests in their original sequential order, which happens to have good spatial locality – total execution time balloons to 42 937 cycles ($4.5\times$ worse). This counterintuitive result demonstrates that reordering and parallelism are more important than raw row-hit rate: FRFCFS can overlap precharge–activate sequences for one bank with data transfers on another, hiding latency through bank-level parallelism. The serializing proxy eliminates this overlap, turning every DRAM timing constraint into a sequential stall. The lower per-request latency (61.90 cycles) is an artifact of the simplified model, not a genuine advantage.

Task 3: Address Mapping

Switching to ChRaBaRoCo mapping is catastrophic: cycles jump to 78 845 and average read latency to 1844.21 cycles. In this mapping, the row bits are placed below the bank bits in the address decomposition, which means that consecutive addresses map to *different rows within the same bank*. Each successive access therefore triggers a row-buffer conflict – requiring a precharge of the current row, activation of the new row, and then the column access – rather than a row-buffer hit. The row-hit rate only marginally increases to 55.9% (from 53.9%) because a few accesses still happen to land in the same row, but the conflict count rises from 707 to 878, and the near-elimination of misses (down to 7) means almost every non-hit access is a costly conflict rather than a cold miss. This result underscores why address mapping is one of the most impactful microarchitectural decisions in memory controller design.

Task 4: Channels and Timing Sensitivity

Adding a second channel cuts total cycles nearly in half to 4 889, the best throughput of all configurations. The row-hit rate drops to 32.7% because the address interleaving across channels splits spatially local requests between two independent controllers, reducing the per-channel row-buffer locality. However, the throughput gain from parallel channel operation more than compensates: two channels can service two independent requests simultaneously, and the bank-level parallelism within each channel is preserved.

Timing parameter	Cycles	Avg read lat.	Increase from baseline
Baseline	9 594	262.9	–
tRCD: 22 $\rightarrow$ 44	10 534	291.9	+9.8%
tRP: 22 $\rightarrow$ 44	10 625	281.9	+10.7%
tRAS: 52 $\rightarrow$ 104	10 003	269.0	+4.3%
tCCD: 4 $\rightarrow$ 8	12 203	322.1	+27.2%

Table 10: Part D Task 4 – timing-sensitivity experiment.

The timing-sensitivity experiment reveals that tCCD (column-to-column delay) is the binding constraint for this trace: doubling it from 4 to 8 cycles increases total execution time by 27.2% and average read latency by 22.5%, far exceeding the impact of doubling tRCD (+9.8%), tRP (+10.7%), or tRAS (+4.3%). This is because the trace has substantial row-buffer locality (53.9% hit rate), meaning that many consecutive accesses hit the same open row and are gated only by tCCD – the minimum spacing between successive column commands. Doubling tCCD therefore directly throttles the sustained throughput for these hit sequences. In contrast, tRAS has the smallest impact because it constrains the minimum row-open time, which is rarely the bottleneck when rows are kept open for multiple column accesses.

Part E – gem5: Whole-Program Simulation

Central question: does the program you care about actually run faster with STT-MRAM?

Part B gives the SRAM configuration a 7-cycle L2 hit latency (derived from the 2.90 ns CACTI access time at a ~ 2 GHz clock), while Part C shows that the same die area occupied by 2 MB of SRAM can hold ~ 8 MB of STT-MRAM, with a 14-cycle hit latency (from the 1.591 ns NVSim read latency, accounting for the $2\times$ slower access). The gem5 runs therefore compare 2 MB / 7-cycle SRAM against 8 MB / 14-cycle STT-MRAM at graph scale -g 14, using atomic warm-up followed by O3 (out-of-order) execution.

Kernel / Config	IPC	L2 miss rate	simSeconds	simInsts
bfs / SRAM (2 MB, 7 cyc)	0.3335	16.64%	0.43711	291 572 930
bfs / MRAM (8 MB, 14 cyc)	0.3392	7.78%	0.42985	291 572 930
sssp / SRAM (2 MB, 7 cyc)	0.3158	31.46%	0.54299	342 904 674
sssp / MRAM (8 MB, 14 cyc)	0.3313	9.50%	0.51754	342 904 674

Table 11: Part E Task 1 – gem5 comparison using identical instruction streams.

Task 1: Whole-Program Comparison

STT-MRAM wins on both kernels. For bfs, the speedup is $1.017\times$ (1.7%); for sssp, it is $1.049\times$ (4.9%). Since the instruction stream is identical between configurations (same binary, same inputs), the performance difference comes entirely from the L2 cache behavior. The larger MRAM cache captures a significantly larger fraction of the working set, reducing the L2 miss rate from 16.64% to 7.78% for bfs and from 31.46% to 9.50% for sssp. Each avoided L2 miss saves a full DRAM access ($\sim 100+$ cycles), while each L2 hit only costs an extra 7 cycles ($14 - 7$) compared to the SRAM configuration. The net effect is positive because the miss-rate reduction converts many expensive DRAM accesses into cheaper (albeit slower) L2 hits.

Kernel	Speedup	IPC change	Miss-rate change	Reason
bfs	$1.017\times$	+1.7%	16.64% $\rightarrow$ 7.78%	Moderate locality benefit
sssp	$1.049\times$	+4.9%	31.46% $\rightarrow$ 9.50%	Large capacity offsets slower hit

Table 12: Part E – derived comparison. Speedup > 1 means MRAM is faster.

Task 2: Kernel Sensitivity

The sssp kernel benefits nearly $3\times$ more than bfs (4.9% vs. 1.7%) because it has fundamentally worse cache locality. With the 2 MB SRAM, sssp suffers a 31.46% L2 miss rate compared to bfs's 16.64%. This means sssp's working set significantly overflows the 2 MB cache, generating many more expensive DRAM accesses. When the cache is expanded to 8 MB via MRAM, sssp captures a much larger fraction of its working set (miss rate drops to 9.50%), converting many DRAM accesses to L2 hits. For bfs, the working

set already partially fits in 2MB, so the additional capacity provides diminishing returns. This result illustrates a general principle: cache-capacity upgrades benefit workloads with poor locality more than those that are already well-served by the existing cache size.

Task 3: CPU-Model Effect

The choice of CPU model materially affects how the SRAM–MRAM trade-off appears. A `TimingSimpleCPU` processes instructions strictly in order, so every L2 access stalls the entire pipeline until the data arrives – making memory latency directly visible in execution time. The `O3CPU` (out-of-order) can overlap independent memory accesses with useful computation by issuing non-dependent instructions while cache misses are in flight. This means the O3 model partially hides the $2\times$ longer MRAM hit latency by finding other work to do during the extra 7 cycles per L2 hit. Conversely, a `TimingSimpleCPU` would show the capacity benefit of MRAM more starkly because every avoided miss translates directly to saved stall cycles without any overlap. The implication is that the measured MRAM advantage (1.7–4.9%) is a conservative estimate for in-order cores and an optimistic one for aggressively out-of-order cores.

Task 4: Final System Choice

Task	Result	Reasoning
2	MRAM wins for both kernels; sssp benefits more.	Workloads with poorer locality gain more from capacity than they lose from slower hit latency.
3	O3 hides part of cache-stall cost.	<code>TimingSimpleCPU</code> exposes stalls directly; O3 overlaps independent work, reducing the visible effect of slower hits.
4	STT-MRAM is the better choice, but with caveats.	$4\times$ capacity with $2\times$ slower hits still wins when miss rate drops sharply; write-intensive workloads could reverse this.

Table 13: Part E Tasks 2–4 – interpretation summary.

The simulation chain points to STT-MRAM as the preferred L2 technology for this specific workload and configuration: Part C establishes that the same die area yields $4\times$ the capacity, Part D characterizes the DRAM backend that penalizes every L2 miss, and Part E shows that the larger cache’s miss-rate reduction (-53% for bfs, -70% for sssp) outweighs the $2\times$ hit-latency penalty on both kernels. However, three legitimate reviewer attack points remain: (i) the $4\times$ capacity ratio assumes the full SRAM area converts to MRAM at NVSim’s density, which may be optimistic if peripheral circuitry does not scale; (ii) the O3 CPU model hides some of the hit-latency cost, so a simpler core would change the balance; and (iii) graph scale `-g 14` produces a smaller working set than the assignment’s suggested `-g 18`, which would stress the 8MB MRAM cache more and potentially shift the outcome.

The cross-part logic is internally consistent. CACTI supplies the 2MB SRAM hit baseline, NVSim supplies the density–latency trade-off, Ramulator characterizes the DRAM penalty for misses, and gem5 applies these numbers to identical instruction streams. MRAM loses on hit time but wins on miss rate, especially for sssp; whole-program timing therefore favors the larger cache under this exact 45 nm, DDR4, FRFCFS scheduler, and O3 CPU setup.

Conclusion

SRAM remains the superior bitcell in isolation – it is faster, lower-energy per access, and simpler to integrate. However, at the system level, STT-MRAM wins when its density advantage ($\sim 4\times$ more capacity per unit area) can be leveraged to reduce cache miss rates significantly. The main bottlenecks that determine which technology “wins” are cache locality of the target workload, the DRAM row policy and address mapping (which set the cost of each miss), and the CPU model’s ability to hide memory-access

latency. Device-level speed alone does not decide the answer; the system context is what tilts the balance.